

Calibração de câmera com padrão planar

CI1026 - Visão Computacional e Percepção — TA02 código: <https://github.com/matiasedd/camera-callibration>

1. Introdução

A ideia do trabalho é "ensinar" o computador como a nossa câmera enxerga. Uma câmera de verdade não é aquele modelo de furinho perfeito dos slides: a lente entorta as retas perto das bordas e o centro da imagem não é exatamente o centro óptico. Calibrar é descobrir os números que descrevem isso: a matriz K (distâncias focais f_x , f_y e o ponto principal c_x , c_y), os coeficientes de distorção e , para cada foto, onde a câmera estava (rotação + translação).

O jeito que usamos é o clássico do Zhang (2000): tira um monte de foto de um xadrez de tamanho conhecido, cada uma de um ângulo. Em cada foto dá para casar "esse canto do mundo foi parar nesse pixel", e juntando tudo o OpenCV resolve os parâmetros minimizando o erro de reprojeção (quantos pixels separam o canto detectado de onde ele deveria cair). É o que o `cv2.calibrateCamera` faz por dentro, basicamente seguimos o tutorial oficial.

2. Método

- **Câmera:** Samsung Galaxy A24 (a traseira), fotos 4080x2296.
- **Padrão:** o tabuleiro grande do VRI, 10x5 quadrados de 90 mm de lado, que dá 9x4 cantos internos. O mundo fica com $Z=0$ no plano do tabuleiro, origem no primeiro canto interno, unidade em mm.
- **Fotos:** 20, mexendo no ângulo e na distância, sempre com o tabuleiro inteiro aparecendo.
- **Pipeline:** `findChessboardCornersSB` para achar os cantos (o detector normal falhava em algumas, o SB achou em todas) -> `calibrateCamera` no modelo de 5 coeficientes (k_1 , k_2 , p_1 , p_2 , k_3). Cada etapa é um script numerado no repo, de 01 a 06.



Figura 1: uma das 20 fotos com os 9x4 cantos internos detectados (script 01)

3. Resultado da calibração

As 20 fotos serviram. O que saiu:

Matriz da Câmera (mtx)				Coeficientes de Distorção (dist)				
[[fx, 0, cx], [0, fy, cy], [0, 0, 1]]				[k1, k2, p1, p2, k3]				
	X	Y	Z	k1	k2	p1	p2	k3
X	3201.51	0.00	2027.74	0.3544	-2.0277	-0.0097	0.0056	3.3866
Y	0.00	3220.66	1059.25					
Z	0.00	0.00	1.00					

Figura 2: matriz intrínseca e coeficientes de distorção obtidos (script 02)

```
K = [[3201.5    0.0   2027.7]
      [  0.0 3220.7  1059.3]
      [  0.0    0.0    1.0]]
dist = [k1    k2    p1    p2    k3 ]
       = [0.354 -2.028 -0.010 0.006 3.387]
RMS de reprojeção = 2.66 px
```

Coisas que notamos:

- $f_x \approx f_y$ (0.6% de diferença): pixel quadrado, certinho para celular. $f \approx 3200$ px numa imagem de 4080 de largura dá uns 65 graus de campo na horizontal, o que fecha com câmera principal de celular.
- c_x caiu quase no centro (2040); o c_y ficou uns 90 px acima do meio. Estranho, mas pelo visto é normal em calibração real.
- O RMS de ~2.7 px assusta na primeira leitura, mas numa imagem de 4080 px é ~0.07% da largura. Projetado de volta no tabuleiro dá coisa de 1-3 mm, o que provavelmente é o próprio tabuleiro empenando (é papel sobre placa) mais um leve blur em algumas fotos.
- Por curiosidade testamos o modelo racional ($k_1..k_6$): o RMS praticamente não mexeu e os coeficientes vieram absurdos ($k_2 \sim -760$). Overfit claro, ficamos com o modelo de 5 mesmo, que é o que o tutorial usa.

4. Remoção da distorção

Com K e $dist$, `getOptimalNewCameraMatrix + cv2.undistort` remapeiam a imagem para uma "câmera pinhole ideal". O script 03 gera para cada foto um painel com original, corrigida e um mapa de calor da diferença. O mapa de calor é a parte que achamos mais legal: dá para ver que a correção é forte nas bordas e quase zero no centro, que é exatamente o que a teoria de distorção radial diz.



Figura 3: original x corrigida x mapa de calor da correcao (script 03)

Mas olhar a foto não prova muita coisa, então fizemos um teste melhor: uma fileira de cantos do tabuleiro é **reta no mundo**, então na imagem ideal os cantos detectados tinham que cair em cima da reta entre as duas pontas. Na original eles desviam até **8.3 px** da corda (dá para ver o arco); na corrigida o máximo cai para ~3 px e a maioria fica ~1 px, que é o próprio erro de detecção, não mais distorção (script 06).

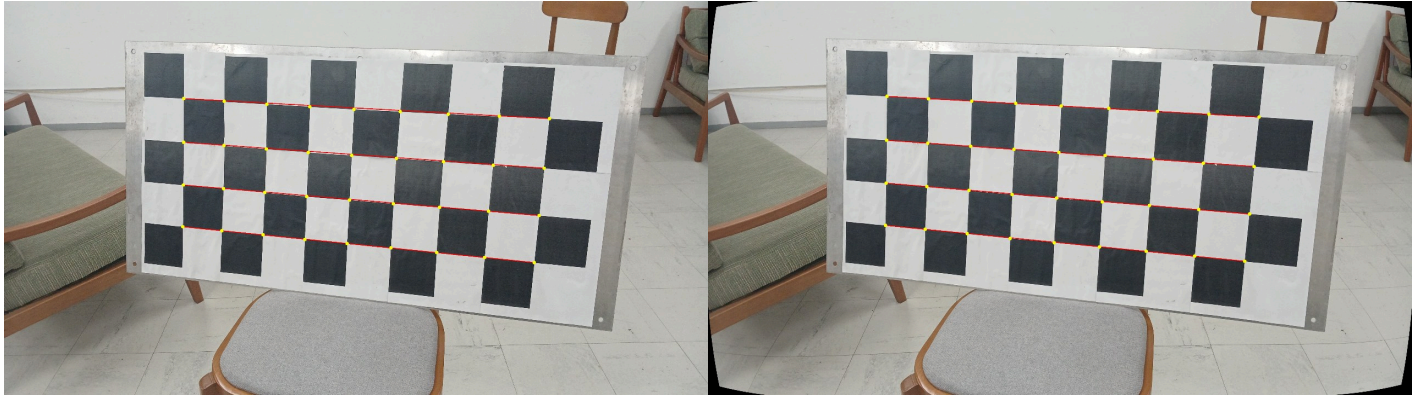


Figura 4: fileiras de cantos (colineares no mundo) x corda entre extremos: antes curvam, depois alinham (script 06)

5. Experimento: 3D -> imagem

O enunciado pede: dada a posição de pontos no espaço 3D, achar as coordenadas deles nas imagens, e conferir.

Como cada foto tem $rvec/tvec$, qualquer ponto (X, Y, Z) do referencial do tabuleiro vira pixel com `cv2.projectPoints` (script 04). Fizemos dois testes:

(a) Conferência numérica. Os cantos internos são pontos 3D conhecidos, então projetamos eles de volta em cada foto e comparamos com os cantos detectados. O erro médio fica ~2-3 px por imagem (máximo ~9 px). É o próprio RMS da calibração aparecendo de novo, o que fecha a conta (se desse muito diferente teria algo errado).

(b) Conferência visual. Projetamos coisas 3D que *não* entraram na calibração: um cubo de 90 mm de lado "em pé" sobre um dos quadrados (Z negativo = para fora do plano, na direção da câmera), os eixos do mundo de 270 mm e um ponto flutuando 180 mm acima do centro (a estrela). Se o modelo estiver certo, o cubo assenta em cima do quadrado visto de qualquer ângulo. E assenta:



Figura 5: projecao 3D->2D: cantos projetados (amarelo), cubo de 90mm em pe (vermelho), eixos do mundo, ponto flutuante (estrela)

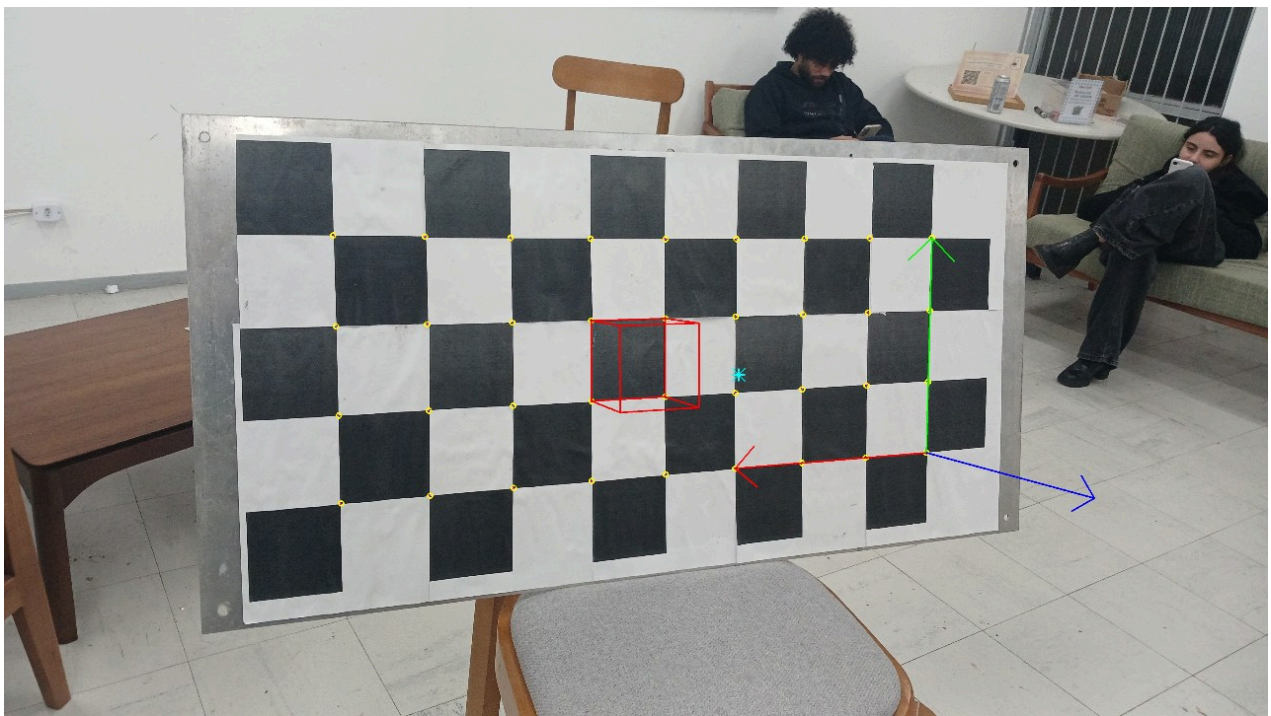


Figura 6: mesmo experimento em outra view — o cubo continua assentado no quadrado

Confesso que essa parte foi a que mais nos convenceu de que estava certo: os números podiam ter saído errado igual em todo lugar, mas o cubo grudado no quadrado em fotos de ângulos diferentes é difícil de fingir.

6. Bônus: "estéreo" improvisado

Não tínhamos duas câmeras, mas duas fotos da mesma câmera em posições diferentes funcionam como par estéreo. Como cada foto já tem a pose no referencial do tabuleiro, montamos $P = K[R|t]$ para as duas views, tiramos a distorção dos pontos 2D e usamos `cv2.triangulatePoints` para reconstruir os cantos em 3D (script 05).

Para conferir é direto: a posição real de cada canto já conhecemos. Num par de views com ângulos bem diferentes deu:

```
erro médio de triangulação: 1.17 mm (máx 2.68 mm, quadrado = 90 mm)
canto 0: real (0,0,0) -> triangulado (-0.1, 0.7, -1.3)
canto 17: real (720,90,0) -> triangulado (720.1, 89.9, 1.2)
canto 35: real (720,270,0) -> triangulado (719.5, 270.1, -0.4)
```

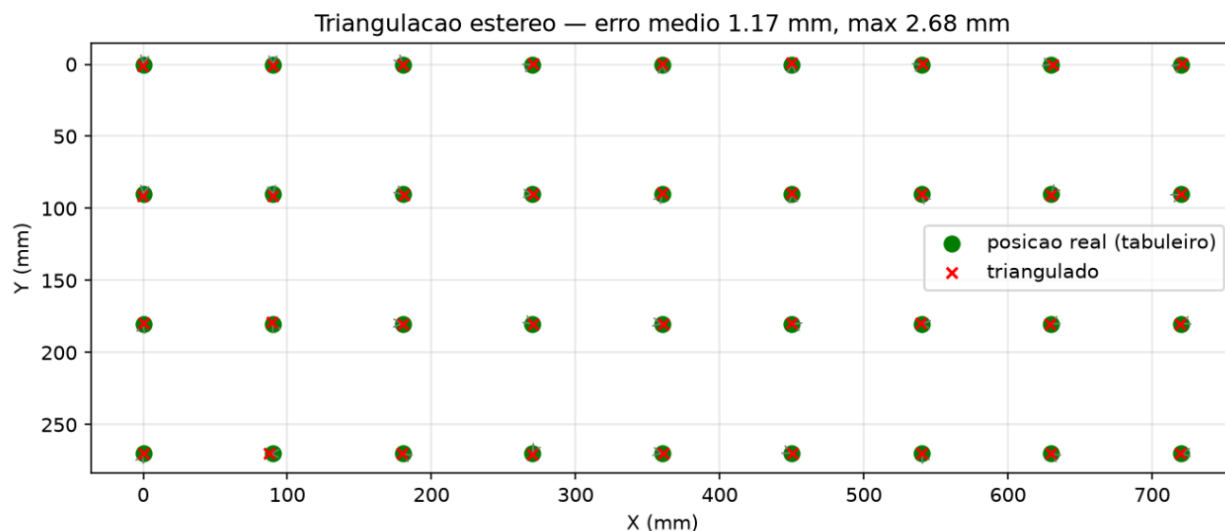


Figura 7: vista de cima do tabuleiro: posicao real dos cantos x triangulada por 2 views (script 05)

~1 mm de erro num tabuleiro de 810 mm, com foto de celular. Esperava bem pior, sinceramente.

7. Discussão / problemas

- As fotos ficaram meio "parecidas": quase todas na mesma distância e com o tabuleiro no centro. As bordas da imagem ficam pouco observadas, o que é provavelmente o motivo dos k_2/k_3 saírem altos (eles se compensam e só divergem onde não tem dado nenhum). Se refizesse, botaria o tabuleiro mais nos cantos das fotos.
- O tabuleiro não é perfeitamente plano; se empina 1-2 mm isso vira residual direto. O RMS não desce abaixo do quanto o padrão é "honesto".
- O detector SB salvou: o `findChessboardCorners` normal falhava com reflexo e ângulo feio, o SB achou nas 20. E a precisão subpixel dele é o que deixa a triangulação bater ~1 mm.
- A medida de 90 mm só muda a escala das poses e da triangulação; K e dist não dependem dela. Sorte, porque a régua não era das melhores.

8. Conclusão

O pipeline do tutorial do OpenCV funcionou de ponta a ponta: calibramos a câmera, tiramos a distorção (com prova numérica, não só no olho), projetamos pontos 3D arbitrários nas imagens conferindo contra os cantos detectados, e ainda fechamos uma triangulação estereoscópica com erro de ~1 mm. Para a primeira vez mexendo com isso, ficamos satisfeitos: o cubo 3D assentando no quadrado em qualquer ângulo e o scatter real x triangulado se sobrepondo foram a confirmação de que as matrizes estavam certas.